# Part 01 – Questions & Answers

### Q1: Why code against an interface rather than a concrete class?

It decouples your code from specific implementations, depending only on behavior contracts. This allows you to swap classes (e.g., replacing **Car** with **Bike**) without breaking dependent code, simplifies unit testing via mocks, and supports the Open/Closed Principle.

### Q2: When to prefer an abstract class over an interface?

Use an abstract class when closely related classes share state (fields) and actual implementation code (like concrete base methods). Use an interface when defining an independent capability or contract that unrelated classes can implement, especially when multiple behaviors are needed.

### Q3: How does implementing IComparable improve sorting?

It lets a custom type define its own natural sorting order via **CompareTo**. Built-in .NET utilities like **Array.Sort()** or **List<T>.Sort()** can then sort collections automatically without needing custom sorting logic at every call site.

### Q4: What is the primary purpose of a copy constructor?

To create a new, independent object with the same data as an existing one, preventing unintended side effects with reference types. It ensures that nested reference fields are deeply copied rather than shared.

### Q5: How does explicit interface implementation help resolve naming conflicts?

It allows a class to implement a member with the same name as another member or interface by prefixing it with the interface name (e.g., **void IWalkable.Walk()**). The member becomes accessible only through that specific interface reference, hiding it from the concrete class.

### Q6: What is the key difference between encapsulation in structs and classes?

Both hide fields and expose properties, but they differ in copy semantics. In classes, changes affect the single heap object shared by all references. In structs, assignment copies the entire value, so each instance's encapsulated state remains fully independent.

### Q7: What is abstraction as a guideline, and how does it relate to encapsulation?

Abstraction means focusing on what a type does rather than how, hiding complexity behind a simple interface. Encapsulation is the mechanism (access modifiers) used to achieve that hiding; you cannot have good abstraction without encapsulation to back it up.

### Q8: How do default interface implementations affect backward compatibility?

Introduced in C# 8.0, they allow developers to add new members to existing interfaces without breaking implementing classes. Existing classes automatically inherit the default behavior, avoiding compilation errors.

### Q9: How does constructor overloading improve usability?

It lets a class provide multiple ways to initialize an object (e.g., **Book()**, **Book(title)**) depending on available data. This avoids helper methods and applies sensible defaults for missing info.

# Part 02 – Questions & Answers

### Q1: Summary of an abstract class LinkedIn article concept:

An abstract class cannot be instantiated directly and serves as a base class. It combines fully implemented members with abstract members that derived classes must override. It allows code sharing and state management, limited by C#'s single-inheritance rule.

### Q2: Coding against an interface vs. coding against abstraction:

Coding against an interface means depending on a behavior contract rather than a concrete type. Coding against abstraction is the broader principle (covering abstract classes too) ensuring code relies on stable, general concepts rather than volatile implementation details.

### Q3: Abstraction as a guideline and its C# implementation:

It means exposing only necessary features while hiding internal complexity. In C#, this is implemented via interfaces (**IVehicle**, **IShape**), abstract classes (**Shape**), private fields with public properties, and method overriding.

## Part 03 – Bonus & Self Study

### Operator Overloading:

Allows custom types to redefine built-in operators (**+**, **-**, **==**) using the **operator** keyword. This lets instances combine or compare using natural syntax (e.g., **vectorA + vectorB**) rather than named method calls.

### 1. Memory Allocation:

Stack: Stores local variables and method frames; fast, Last-In-First-Out, cleared automatically when out of scope. Heap: Stores reference types (classes) managed by the Garbage Collector. Value types live where declared, while reference types hold a heap pointer.

### 2. Dependency Inversion Principle (DIP):

High-level modules should not depend on low-level details; both should depend on abstractions. Using interfaces (like **ILogger**) ensures that swapping implementations doesn't require rewriting high-level logic.

### 3. Specification Design Pattern:

Encapsulates complex business rules into dedicated small classes implementing a common interface (e.g., **ISpecification<T>**). It allows rules to be tested independently and combined (using AND/OR logic) without scattering **if** statements across the codebase.